Kindred Siege design development and production research

Applying five video essays to a playable and testable roguelike

Prepared for Jacob and the Kindred Siege development team

11 October 2026 | Version 1 | Critical source review and practical production proposal

This paper preserves the current design locks. Recommendations are proposals, not new owner decisions. Its implementation snapshot is dated 11 October 2026 and includes Claude's latest recorded 1x world and camp pass.

Abstract

Kindred Siege has a coherent opportunity: a fast roguelike in which automatic battles are influenced by player orders, defeated tribes provide surviving recruits, and chiefs change their behaviour because of recorded previous fights. The immediate challenge is making those relationships understandable and rewarding inside a complete repeatable run. Expanding the roster before proving them would multiply content costs while leaving the central design risk unresolved.

This paper critically analyses five user-selected videos about autobattlers and tactics games, triangulates their arguments with developer publications and research, and applies the findings to KS's current locks, source and handoff evidence. The review finds support for readable unit interactions, constrained choices, gradual introduction of complexity, and testing progression as well as isolated battles. It rejects several literal transfers: a competitive shared shop, obligatory merging, expiring recruits, wholesale turn-based combat, and unrestricted procedural classes.

The recommended slice joins combat, recruitment, a useful camp purchase, a changed rematch and an inspectable record of the change. It also includes the locked first Alpha encounter. Development should establish authoritative gameplay explanations and durable history; production should measure accepted, integrated content and observed player understanding. A proposed experiment plan separates correctness, combat balance, presentation and player experience. No new playtest, causal effectiveness result, commercial forecast or novelty certification is claimed by this paper.

Executive recommendation

Make the player understand and enjoy one complete cycle before expanding production across 24 tribes. That cycle should let a new player protect an army, break a chief's nerve, gain survivors and a chief order, use that reward in a subsequent fight, return to a useful camp, and recognise one evidenced adaptation on a later run.

KS already has many of the parts. The gaps are in their connection. A material counter does not fulfil an upgrade loop without a purchase and a visible effect. A learning log does not fulfil the player's experience of rivalry without a recognisable rematch. A damage summary does not demonstrate why another choice would have worked. A large animation folder does not demonstrate readable combat at the approved scale.

The next production milestone should therefore be a complete small loop with evidence, followed by broader roster work. Preserve the full-game counts and identities. Restrict the next delivery to the authored slice, without treating its smaller content set as a reduction of the eventual game.

Contents

1 Research questions and method 3
2 The five sources and their limits 4
3 What the sources support together 6
4 The current KS design and implementation boundary 7
5 Design the experience around three decisions 8
6 Make rivalry observable and bounded 10
7 Connect camp progression to the next run 11
8 Progression inside the locked roster 12
9 Combat presentation and animation production 13
10 Development architecture and trustworthy explanations 14
11 Procedural design without uncontrolled scope 15
12 Evaluation plan 15
13 Production plan and ownership 17
14 Estimate content cost before expanding it 19
15 Principal risks and decision register 19
16 Conclusions and limits 20
References 21
Appendix Practical review checklist 22

1 Research questions and method

The review addresses five questions. Where should player agency sit when units fight automatically? How can KS offer immediate spectacle without making outcomes obscure? What should persist across runs so that losing still leads somewhere? How can procedural rivalry create meaningful variation inside a bounded authored roster? What must the team verify before increasing content production?

The primary review corpus is the five videos supplied by Jacob. Their English caption transcripts were exported from YouTube and read in full, including critical qualifications near their endings. The first four exports identify their captions as automatic; the fifth says authored or unspecified. Names and publication dates were checked against the visible video pages. Timestamp citations are navigation aids and may precede or follow a sentence slightly because captions are segmented.

This is a qualitative, purposive review, not a systematic survey of every autobattler and not an experiment. It analyses spoken arguments and descriptions. It does not claim continuous audiovisual viewing, frame-by-frame animation analysis, or independent play of every referenced game. Visual production recommendations below come from KS's own constraints and are identified as proposals, rather than invented observations of the videos.

Each source was read for six themes: allocation of decisions, intelligibility of combat, sources of variation, consequences of loss, progression and production lessons. For each theme, the review asks whether the underlying mechanism fits KS, what it would cost to adopt, and what evidence could disprove the recommendation. Agreement among video creators is useful convergence of opinion, not a measured effect size.

Developer accounts provide supporting evidence for process and mechanics. The MDA paper supplies a framework connecting rules, resulting behaviour and player experience [R1]. Riot's prototype guidance stresses representative play and audience testing before production expansion [R2]. Riot's analysis-team account distinguishes simulation evidence from the pacing information obtained in playtests [R3]. These are contextual methods, not guarantees of success for KS.

Evidence categories

The paper distinguishes owner locks, observed source, recorded validation, interpretation and proposal. Owner locks specify intended behaviour. Observed source establishes that a particular implementation exists at inspection time. Recorded validation reports what a dated handoff tested; it is not a new independent rerun. Interpretations connect evidence to a design concern. Proposals remain subject to owner selection and testing.

The repository was inspected read-only on branch game-one at commit bbf1f2b, with substantial uncommitted work. A commit identifier alone cannot reproduce that working state. The research evidence folder retains hashes and snapshots of the selected local sources. No Unity editor or player was launched, no owner save was opened, and no fresh balance or usability result was produced for this paper.

2 The five sources and their limits

Source	Author and publication	Main contribution to this review	Evidential limit
V1 Game design deep dive what makes autobattlers fun	Savallion, 25 June 2023	Team construction, economy, readable roles and progression across several games	Personal critical play; some games were first impressions and versions have changed
V2 History of Autobattlers	Skoottie, 7 June 2022	Drafting, iteration between fights, and recognisable presentation as an entry point	Historical overview and accessibility argument, not a causal study of commercial success
V3 Autobattlers 101 An Intro To The Genre	Your Friendly Neighborhood Gamers, presented by Dylan, 11 December 2024	Different genre forms and progressive introduction of systems	Introductory recommendations; descriptions should not be treated as current exhaustive rulebooks
V4 Every Auto Battler has the Same Problem	Gamedev Greedable, 2 October 2026	Resource turnover, the reward for loss, and testing a complete mechanic	Developer account of an unreleased game; broad genre claims overreach the examples
V5 Good Game Design Tactics Games	Snoman Gaming, 9 March 2019	Consequences, interacting options, objective design and speed of play	Criticism from an explicitly new tactics player; slower games have different attention budgets

V1 Constructing a team must create consequences

Savallion foregrounds synergies and resource allocation in TFT, then contrasts that complexity with the small action vocabulary of Super Auto Pets. The later examples discuss unclear role descriptions, weak combat feedback, preserving experienced units and planning ahead. The closing diagnosis of the creator's own game is especially relevant: selecting different units needs to change what the team can do. See V1 at 03:45, 11:36, 14:44 and 31:30.

For KS, this supports understandable combinations of class, position, equipment and orders. It does not require a shop or a count-three-units trait bonus. Existing target access and Warden protection already create interactions worth exposing. The design test is whether replacing a fighter changes a useful decision, not merely whether its damage total changes.

The preference for predictable battle resolution around 12:09 is an opinion, not a universal rule. KS can preserve reproducibility while retaining seeded uncertainty. Reproducibility means the same recorded inputs reproduce the outcome; predictability means the player can anticipate it. They are related but different

requirements. The video's opening historical shorthand about TFT should also not be treated as proof that it originated the genre.

V2 Familiar presentation lowers the cost of understanding

Skoottie connects drafting to constructing an incomplete team from restricted availability, then improving it between battles. The later argument contrasts the visual familiarity available to established franchises with Super Auto Pets' immediately readable animals and collisions. See V2 at 00:20, 02:06 and 07:43.

KS cannot assume players recognise its tribes, species or equipment. Its first impression therefore needs familiar functions inside original designs: protection should look protective, a distant attack should visibly travel, healing should identify its recipient, and a chief should be easy to locate. The player's first useful vocabulary can be shield, shoot, heal and break nerve. Detailed lore can follow successful recognition.

The video's account does not demonstrate that familiar characters caused the success of the cited games. Distribution, existing audiences, pricing and production quality are not controlled. The practical conclusion is narrower: test whether unfamiliar players can read KS without prior lore knowledge. Do not infer that the game needs licensed familiarity, simpler aesthetics or a different premise.

V3 Teach complexity over time

Dylan presents several variations on the genre. Tales and Tactics is described as introducing additional encounters and mechanics through progression, while Super Auto Pets allows planning at the player's pace. Unicorn Overlord illustrates that automatic squad resolution can sit inside a wider strategy RPG. See V3 at 02:39, 03:34, 04:30 and 06:30.

The transfer to KS is progressive disclosure. Players can learn one order, then target access, then a chief's distinctive threat, before needing the complete class catalogue. The camp can introduce an upgrade when its use has become understandable. A fixed final scope does not require presenting all its systems on the first screen.

This source does not settle KS's tier distribution or class unlock schedule. Its descriptions of particular modes, lives and seasonal content are dated examples. Borrow the teaching principle, not those values. KS's active directives also mean it cannot simply inherit the uninterrupted planning time of an asynchronous shop phase.

V4 Loss needs a complete return

Greedable argues that persistent accumulation can leave a successful unit with few remaining decisions. Fruit Auto Fighters introduces ageing fruit, composting and power that persists in occupied slots. Crucially, the creator reports that testing before the equipment and fertilizer payoff existed made the game feel shallow. The closing qualification admits that accumulating power remains, but has moved elsewhere. See V4 at 04:03, 07:11, 13:10 and 15:04.

The universal claim about autobattlers not losing units is too broad. V1 itself discusses attrition in Despot's Game; its publisher describes sacrificing a squad through procedural dungeons [R8]. The source's fixed-sounding loss-aversion multiplier is not adopted as a KS parameter. The Steam page identifies Fruit Auto Fighters as unreleased at access time, so this is a development hypothesis rather than proven commercial evidence [R7].

KS already resets recruits, gear and chief orders between runs. It should not add rot, mandatory sacrifice or another currency to imitate this video. The useful application is to finish the existing return: preserved materials must buy something useful, and accumulated knowledge must change a visible future encounter.

V5 Objectives and interactions create tactical depth

Snoman's examples emphasise how targeting, movement, survival and environmental interactions alter decisions. Into the Breach is used to show why displacement or prevention can matter more than damage, and later passages discuss speeding up repeated presentation and preventing one strategy from resolving every problem. See V5 at 04:01, 05:16, 07:00 and 08:34.

This is a strong fit for KS's nerve and survivor recruitment. A chief fight can reward keeping enemies alive; Hold the line has a survival objective. These objectives can make protection, interruption and controlled pressure worthwhile without inventing more damage classes.

The transfer has a limit. Turn-based puzzle games allow inspection before every action. KS's fast action requires selective, timely information and brief response windows. Copying a full grid, movement orders, terrain exceptions and a dozen simultaneous predictions would change the experience and create production costs. Preserve the tactical consequence while expressing it through KS's existing formation and directives.

3 What the sources support together

The strongest shared proposition is that depth comes from consequential interactions that the player can understand. None of these sources establishes a required number of classes, a universal number of currencies, or an ideal animation frame count. They instead offer examples of how a restricted set of rules produces different decisions.

There is a useful disagreement about accumulation. V1 values assembling and retaining a satisfying team; V4 values disruption of a settled team. KS should retain the emotional value of recruits during a run and use its already-locked run reset to reopen composition decisions. Rival learning can challenge a successful plan on a later run without deleting a favourite recruit in the middle of the current one. Whether this feels exciting or punitive remains a playtest question.

A second tension concerns speed. V5 praises opportunities to reduce waiting; V1 wants sufficient clarity to understand resolution. KS needs both. Routine attacks should remain quick, while the encounter's important cue must remain readable. Longer source animation is an artistic resource, not permission to delay input or the simulation until every clip finishes.

Riot's Remix Rumble reflection distinguishes useful game-to-game variation from unequal access to resources within the same game [R5]. KS is not a PvP shop game, but the distinction suggests a useful experiment: vary the problem between runs while preserving a comprehensible chance to answer it. Random generation that withholds every answer to a required counter should be counted as a content failure, not celebrated as replayability.

4 The current KS design and implementation boundary

The full-game constraints remain 24 tribes including Alpha, seven tiers, 17 progression-unlocked classes, fast roguelike action, and a 2.5D battle screen. Tiers 1 to 3 carry much of the narrative; tiers 4 and 5 lead toward endgame; tier 6 is pre-endgame; tier 7 is the Sovereign. The exact allocation of classes and tribes remains partly open. Earlier rejected schedules cannot become approved through this paper [L1].

The current run direction crosses one tribe per tier. A tribe presents a few fights and its chief. Recruitment occurs at the chief, with more survivors intended to mean more recruits. Chief orders last for the run. Army, collected gear and chief orders reset at its end; materials, purchased camp upgrades, codex discoveries and learning persist. The four Pals are the surviving inner circle and camp upgraders, not collectible animals [L1].

The following is a scoped source-and-handoff snapshot [L2] [L4]. It is not a replacement for the pending player audit.

Area	Evidence on 11 October	What can be claimed
Combat explanation	CampaignArtWhy.cs contains reach and Warden messages plus ranked result facts	Explanations exist in source; comprehension and causal completeness are unproven
Tutorial and choice cards	Claude records acceptance of the UI direction; walkthrough and icon cards are present	Design direction is accepted; all later tutorial stops still need human verification
World and camp	Latest Claude entry records separate regional stages and four stations at 1x, with 365 main, 2 reopen and 31 inner-circle checks	Recorded implementation progress, not a fresh independent visual approval
Saving and history	Live CampaignArtLoop.Save still uses File.WriteAllText; isolated repair 224924f provides file recovery and archived fight proof	Backend repair is separately tested; it is not yet integrated into the live player
End of slice	CampaignTribes.All contains Clarth, Bloodridge and Skullpeak; campaign ends after its final available chief	The locked first Alpha hunters still need a design decision and implementation
Camp expenditure	Materials accumulate; no completed first Butcher purchase is established in the inspected evidence	A useful spending loop remains an explicit production dependency
Intertribal learning	Campaign creation seeds WarTaught habits; player fights enter an archive	Seeded prior habits do not establish simulated wars between runs

There is also a rule detail worth resolving before recruitment is promoted as the signature payoff. Campaign.Finish currently tops up recruits to at least three after collecting living enemy fighters. That is an observed implementation detail, not a new owner lock. Measure whether this floor weakens the difference between careful nerve pressure and killing most of the opposition. Jacob should confirm the intended minimum before it becomes a balancing assumption.

The saved-history repair handoff reports 161 passing tests on both .NET and Unity's Mono runtime, plus separate-process verification of 87 fights and 612 lessons. Those results support its scoped backend readiness. They do not establish in-player recovery, hitch-free saving, a historical replay browser or Claude's completion of the hookup [L3].

5 Design the experience around three decisions

Build a team that changes what the player can attempt

The first decision is composition. A fighter should communicate a job, a useful partner and a vulnerability. Class provides the stable combat function; tribe provides a recognisable identity and bounded variation; equipment provides a change the player can see and use. These dimensions should not all modify the same undifferentiated damage statistic.

A useful unit description has two layers. The short layer states an action such as protecting a hurt ally. The expanded layer gives the trigger, recipient, magnitude and limitation from authoritative data. Show the short layer where the player chooses a squad, and the expanded layer on inspection. A lore paragraph cannot substitute for either.

Start with the five slice classes already selected: Warden, Marksman, Skirmisher, Trapper and Medic. Give each an acceptance question. Can a new player identify whom the Warden protects? Can they tell which fighters reach a back-row target? Can they notice a Medic's intervention? Can they describe why a Trapper or Skirmisher changed their plan? A class whose role cannot be demonstrated in the slice should not be disguised by extra particles or adjectives.

Spend an order because the situation makes it valuable

The second decision is intervention. Refilling buttons can create activity without creating agency. A meaningful order needs a recognisable situation, a cost or lost alternative, an observable outcome, and a later consequence. A player who presses Unleash whenever it lights up may be following the best policy the rules currently offer.

Do not solve this by secretly punishing repeated presses. First compare an always-Unleash policy against policies that react to actual tells. If reaction has no advantage, tune the opportunity costs or encounter structure. If reaction is stronger but players cannot see why, improve the explanation. Those are different failures and require different work.

The proposed order roles below are design tests, not new numerical balance.

Order	Situation it should help solve	Tradeoff to make visible	Proof to collect
Hold	An approaching burst or a brief need to protect the line	Holding has an existing nerve cost; remaining passive must not be free safety	Damage prevented, nerve spent and subsequent survival
Focus Fire	A reachable priority target or a deliberate front-line breakthrough	Target access and protection can reduce participation	Eligible attackers, actual target events and kills
Unleash	A real opening where burst or nerve pressure is valuable	Points spent now may remove a timely defence or later cast response	Outcomes compared with a cue-responsive policy on matched seeds
Fall Back	A telegraphed attack that position or timing can answer	Enemies may press a retreat; it must not be universally safe	Cast avoided or reduced and the enemy response
Chief order on key 5	A recognisable advantage earned by beating a chief	Availability and opportunity cost must be understood	The next fight demonstrates a useful application

Parry remains an open mechanic. A first paper prototype could distinguish it from Hold by rewarding a brief, correctly timed response, but window, input, cost, invulnerability and reward need owner selection. It should not become a second mandatory reaction test before the current four orders are understood. Test it separately rather than silently extending the slice's input burden.

Decide how to win the chief fight

The third decision is the kind of victory. KS's premise gives it a useful tension: damage helps win, but dead enemies cannot become survivors. Nerve pressure can therefore reward a different pattern from elimination. This is the most promising connection among combat, collection and the next fight.

Use an explicit worked scenario in a controlled test. One player eliminates a supporting fighter quickly and secures a safer victory with fewer survivors. Another protects their own line, times nerve pressure and ends with more possible recruits. The exact rewards must follow the selected rules. The test asks whether both routes are viable, whether the second is legible, and whether its recruits help in a later encounter.

Avoid explaining this with a new respect bar. The owner dropped that system. The result should show the ending condition, survivors recruited and the chief order gained. The next screen should allow the player to use those rewards. Ending the slice immediately after Skullpeak currently prevents that final reward from receiving a useful trial; the first Alpha encounter is therefore part of the payoff, not merely another boss for the checklist.

6 Make rivalry observable and bounded

The rivalry proposition is stronger when it is stated as a player experience: a chief recognises something that happened, prepares an answer and can be outplayed again. The player should see evidence for the change before concluding that the game simply increased hidden numbers.

Use a compact evidence chain. A completed fight produces a record. A learning rule cites events in that record. A stored habit changes a permitted future policy. An upcoming encounter exposes a clue to that policy. A rematch creates new evidence. This description does not require neural training or an online model; do not label an event-driven adaptation system as machine learning without demonstrating that method.

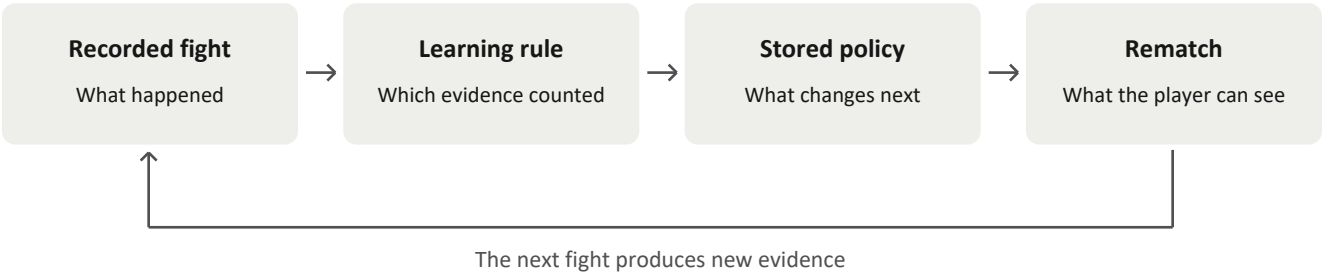


Figure 1 Proposed evidence flow for one rivalry cycle

In a proposed Bloodridge demonstration, the previous fight records repeated successful bursts. A later chief favours a defensive response to that pattern. The preview gives a short warning, and the codex can open the cited fight. The player changes timing or target priority and sees the chief's response fail in a specific moment. This is an illustrative test sequence, not a statement that the whole sequence currently runs in the player.

Bound adaptation with an authored set of responses. A low-tier chief could expose one active learned behaviour in an initial experiment; higher-tier limits can be explored later. That number is a proposed test setting, not a new tier lock. Limit the response's strength, frequency and combinations. A rematch must retain at least one feasible answer using the roster and orders actually available to that player.

Separate recruitment progress from enemy learning strength. If every victory expands the squad and simultaneously teaches every enemy a perfect counter, the player may experience progress as a tax. Conversely, if habits never affect an outcome, the system becomes flavour text. Record both the benefit and the burden and test several consecutive runs, including repeated losses.

Wars between tribes must produce actual evidence

The five originally proposed modules were RegionGenerator, WorldSimulation, ArmyComposer, EncounterBuilder and WorldStore. The handoff explicitly identifies them as a proposal, and WorldStore is persistence rather than random generation [L2]. This paper does not rename them or claim that five complete generators already exist.

Proposed module	Minimum responsibility for the slice	Boundary that contains cost
RegionGenerator	Select authored regional context and compatible presentation parameters	No requirement for a new procedural overworld before the loop works
WorldSimulation	Schedule a bounded tribe-versus-tribe fight at a defined campaign transition	Avoid continuous real-time world processing and uncontrolled history growth
ArmyComposer	Produce legal lineups from approved tribes, available classes and equipment	Reject incompatible combinations; do not invent classes or tribe identities
EncounterBuilder	Assemble a valid objective, army and learning state for a fight	Validate available answers and a clear ending condition
WorldStore	Persist identity, history, compatibility data and provenance	Never replace missing history with invented battles

An initial war experiment could resolve one selected intertribal fight on return to camp, store its seed and commands, and expose one resulting lesson. Frequency and scheduling remain proposals requiring Jacob's call. Use the same battle rules and archive interface as player fights, with provenance identifying the opponent as another tribe. A deterministic battle between policies is evidence of a simulated war; merely preloading a habit labelled war-taught is not.

The research on evaluating generators distinguishes analysis of their output range from asking how players experience that output [R6]. Apply both. Count composition and objective coverage, then ask whether sampled encounters require different decisions. A thousand seeds that all reward the same order are a thousand outputs with little useful variation.

7 Connect camp progression to the next run

The camp should answer three questions at a glance: what came home, what can be changed, and what that change will do next time. This follows directly from KS's locked loss rules. The Pal animations establish personality and recognition, but their stations need a functional transaction for the upgrade promise to exist.

Butcher owns weapons and gear, Glutton armour and body augments, Shade orders, and Whisperer skins and regalia [L1]. Keep these responsibilities distinct. Shared astral materials do not require shared silhouettes, poses or identical work loops. The new open camp layout should preserve room for the approved activities and tools at the approved 1x world scale.

The first Butcher item is still a decision for Jacob. The following options make that decision concrete without locking a recipe or cost.

Candidate function	Benefit to the slice	Design risk	Required evidence
A visible weapon upgrade with a simple damage increase	Easy to understand and inexpensive to explain	Encourages more killing, potentially weakening survivor recruitment	Player notices it; victory does not collapse into compulsory damage grinding
Gear that adds nerve pressure to an existing attack	Directly teaches the collection premise	Could make Unleash or other nerve tools redundant	A distinct useful role and a meaningful survivor difference
Gear that rewards a properly timed existing order	Links the camp to tactical mastery	May punish beginners or duplicate a future parry	Clear trigger, understandable feedback and benefit without mandatory perfect timing

The strongest research candidate is the second, provided it fits the existing class and gear rules. It ties expenditure to the game's stated recruitment advantage. This is a recommendation for a bounded prototype, not permission to implement a new effect. An item name, cost, compatible carrier, effect, drawback and persistence policy must be agreed first.

Distinguish a permanently unlocked recipe or station capability from the run's actual carried gear. The locks reset run gear while preserving purchased camp upgrades. Without that distinction, a forge screen can promise permanence and the run reset can appear to delete the purchase. Show the durable unlock and temporary instance separately if that is the selected design.

Do not introduce four new currencies merely because four Pals exist. One legible material pool can support the first transaction while content and balance are tested. More currencies are justified only when they create a desirable decision the player understands. Likewise, cosmetic progression should not quietly become an unannounced combat requirement.

A failed run should return through a short, truthful receipt. Show the materials actually preserved, the discovery actually recorded, and a concise Pal response. Do not award imaginary progress to soften defeat. If a first loss currently earns no useful purchase, test whether the next reachable upgrade is clear and reasonably close; the correct price cannot be inferred from these videos.

8 Progression inside the locked roster

Treat the 17 class definitions as a catalogue of intended roles, not as 17 mandatory roles in every tribe. Making every tribe field every class would imply up to 408 tribe-class pairs before equipment, directional art and animations. That is a theoretical cross product, not the approved roster or an estimate of existing assets.

Use a sparse availability matrix. Each tribe needs an identity, compatible classes, a chief threat, a weakness, a regional context and an art family. Each class needs its signature function and a reason to be chosen. Fill a new matrix cell only when it changes a composition or expresses a necessary tribe identity. This honours the small dataset requirement while retaining the eventual totals.

Teach by behavioural novelty. A new class should introduce one main consideration before combining with established ones. A new tribe can reuse familiar roles while changing their emphasis and chief response. Avoid unlocking a new class, a new resource, a new enemy counter and a new terrain rule in the same first encounter; that makes the cause of failure difficult to identify.

The seven-tier structure does not require 17 classes to divide evenly among six ordinary tiers. Exact allocations remain open. Evaluate a candidate schedule by asking what each unlock enables, whether it has a useful partner already available, whether an upcoming encounter lets the player use it, and whether rivalry still offers accessible responses. Do not resurrect rejected numerical schedules in the name of coherence.

The codex should distinguish discovery from current ownership. Joining during a run records a permanent tribe discovery; it does not imply permanent possession of that army. A useful codex entry can contain the tribe's recognised identity, encountered roles, acquired knowledge and an optional history link. Collection should reward attention and experimentation rather than force repeated arbitrary chores.

9 Combat presentation and animation production

The current presentation lock is specific: world, stages, fighters and camp at 1x; menus and text on the 2x kit. Preserve it. The new per-region stages are recorded progress, while a fresh visual audit still needs to test the foreground hierarchy and room for the maximum intended formation. A change in stage art must not accidentally imply cover, walkable lanes or terrain effects that the simulation lacks.

Allocate attention by importance. The player's selected target, a chief's imminent cast and a nerve break deserve priority. Routine damage can remain plentiful but subordinate. When many numbers occur together, merge related low-priority values and preserve the important event's identity. Colour should be reinforced by shape, position or a short label, particularly for guard, hit, cast and break states.

The useful animation contract is expressed in events: anticipation, release or impact, recovery, and return to a valid state. The simulation owns whether an attack occurred and what it did. Presentation maps the event to an appropriate clip and effect. A 40-frame source clip can contain an extended flourish without forcing a 40-frame delay before the player may respond. Exact durations depend on the chosen playback rate and combat cadence.

Idle loops need continuity; attacks need identifiable contacts; hit reactions need interruption rules; death needs an unmistakable ending pose; block and a future parry need distinguishable signals. These requirements do not imply the same length or motion amplitude for every class. A Marksman, Warden and chief should not become interchangeable because their exports share a frame count.

The approved camp behaviours also have different purposes. Butcher works with equipment; Glutton sleeps; Shade studies while levitating; Whisperer shapes her dark veil while levitating. Highlighting should make the relevant task readable without forcing the other three into distracting full-intensity loops. The live camp pass must be judged at its actual scale, not only in a large browser sprite preview.

A production asset is more than an exported strip

For every asset family, retain the approved reference, PixelLab generation receipt where relevant, editable Aseprite master, palette, canvas size, pivot, direction, frame timing, event markers and intended runtime use. Record separate states for generated, cleaned, owner-approved, integrated and checked in the player. No state should be inferred from the presence of a file.

Use one representative unit to establish the import and animation contract before multiplying a new action across a tribe. For an existing approved batch, audit its actual coverage instead of regenerating it by default. Missing assets need named requests: character, action, facing, canvas, usage and approval status. A label saying art pending is useful in an internal test but fails Jacob's requested playable art milestone.

A short sound pass belongs in the readability milestone. Propose a small vocabulary for order accepted, order unavailable, shield contact, chief tell, nerve break and reward receipt. Coordinate it with the visible cues, permit volume control, and avoid simultaneous sounds masking the chief tell. This paper neither supplies audio nor claims that a soundless prototype establishes the final combat feel.

10 Development architecture and trustworthy explanations

Keep authoritative combat, campaign state and presentation separate. The frontend sends commands and displays the resulting state. It should not reconstruct rules independently, because a duplicated target or victory rule can become wrong after balance changes. The existing CampaignArtWhy.cs contains local reach and ending logic; that is a maintenance risk to review, not proof that it is currently incorrect.

A proposed backend query for a target preview should expose eligible attackers and a reason for each exclusion. An encounter result should expose the authoritative ending reason and rewards. A learning record should expose the originating fight and event range. These are conceptual interface responsibilities, not mandated new class names or permission to replace Claude's API.

Separate three meanings of explanation. A factual statement reports an event, such as a Warden absorbing hits. An attributed lesson states which events caused an adaptation rule to fire. A counterfactual asks whether changing a decision would change the outcome. Only the third requires a controlled alternate simulation. Do not present a ranked damage or healing total as mathematical proof that it caused the win.

The current result panel ranks damage, healing, guarded hits, cast responses and other facts with starting weights. That is useful feedback. It becomes misleading if its heading implies a fully established causal explanation. Prefer wording that remains true to the evidence, then make deeper inspection optional. A future alternate replay should freeze the original seed, initial state and rules, record the one changed command, and explain that the alternate outcome is a simulation under those conditions.

Persistence is part of the design claim

The rivalry promise spans sessions, so its evidence must survive closing the game. The isolated persistence work already addresses the backend requirements: a validated temporary write, replacement with a last-good backup, explicit recovery, versioned rules, archived commands and events, and verification before resuming active combat [L3]. The next work is integration and player verification rather than another parallel save implementation.

Claude's handoff step includes both loading entry points as well as the write call. A primary-only file existence check can hide a recoverable backup. Recovery notices must remain visible, and a failed load must not silently become an empty new camp. Old saves cannot recover fight records they never stored. Compatibility policy must make that absence explicit.

Define archive retention before making long-term learning claims. If history is capped, preserve the evidence for retained lessons or clearly mark a lesson's source as unavailable. Measure save time at realistic campaign sizes and with the intended autosave cadence. The earlier local storage timings are useful evidence but do not guarantee smooth Unity frames during a fight.

11 Procedural design without uncontrolled scope

Generation should select and compose approved content within constraints. Its inputs should include tier, tribe identity, available classes, encounter objective, learned policy and player access to responses. Its output should be an inspectable record with a seed and the chosen authored records. A generated name or visual variation cannot substitute for a different tactical situation.

Author validation rules before increasing seed counts. A chief encounter must contain its chief and a valid ending condition. Required counterplay must be available or the encounter must be rejected. An army must satisfy class and equipment compatibility. The stage must match the tribe, and presentation must be able to represent every combatant without an unapproved substitute. Impossible combinations should produce an explicit validation failure.

A small initial test catalogue can span role distribution, target access, protection, burst timing and survival pressure. Vary these dimensions in controlled combinations, then retain seeds that expose meaningful edge cases. Do not attempt exhaustive coverage of every item, fighter trait and learned habit in the first pass. Exhaustive pairings grow quickly, and nominal variety can conceal repeated behaviour.

Rivalry should modify an authored encounter within a readable envelope. It should not secretly replace the objective, erase every weakness and increase every stat at once. Keep a record of which generated difference came from tier progression, which came from a tribe's identity and which came from learning. Without that separation, neither the player nor a tester can tell what the rivalry system contributes.

12 Evaluation plan

The following experiments are proposed. Their sample sizes and acceptance thresholds are starting criteria for development, not published genre standards, promises or completed results. Freeze the build, rules revision, art manifest and seed list for each comparison. Preserve failures alongside successes.

Technical correctness

First verify that a fight replays from the same initial state and commands, including the same ordered events and final checksum. Then verify a complete run through reopen boundaries: before a fight, midway through a fight, after a chief, after a camp purchase and after an intertribal simulation. Check that rewards and learning are applied once and that no unavailable art is substituted silently.

Save testing should include a truncated primary, missing primary with valid backup, unsupported rules, a changed command record and a failed replacement. Use isolated test saves. Retain the original files for diagnosis and do not demonstrate recovery on Jacob's only camp. The player needs a meaningful message when a fight must restart or an archive cannot be verified.

Combat balance

Use a matched set of encounter seeds and starting squads. Compare at least four explicit policies: no orders, repeated Unleash, repeated healer focus, and a policy that responds to the encounter's actual cues. Include a simpler competent policy so that the responsive one is not compared only with deliberately weak opponents.

Measure win rate, time to outcome, order expenditure, remaining nerve, survivors, recruits and objective completion by encounter type. Do not hide a trial failure behind an aggregate win rate. Compare policies at equivalent information access and legal timing. A bot that reads future state or reacts with impossible precision is a diagnostic tool, not an estimate of human potential.

An initial developer run of 100 matched seeds per relevant encounter family is a manageable proposed starting point, followed by targeted expansion when a failure appears. Set a practical superiority threshold before testing and inspect the distribution, not only the mean. If Unleash spam matches the responsive policy everywhere, the design has not yet established the intended need to react. If it wins some easy fights, that alone is not a defect.

Player understanding and enjoyment

Recruit a small formative group spanning newcomers to autobattlers and players familiar with tactics or roguelikes. An initial 8 to 12 participants is a proposed usability pass, not a statistically representative market sample. Let them play without designer coaching, apart from help required to continue after a recorded blocker.

After a selected fight, ask the player to identify the ending condition, one helpful action and one change they would try. Record the answer before showing the expanded history. During an important target selection, observe whether the player understands reach and protection. At camp, ask what is permanent and what resets. In the rematch, ask what the chief learned and how they know.

Proposed early criteria are that at least 8 of 10 players can identify a fight's ending condition and the camp's reset boundary, and that at least 7 of 10 can explain the demonstrated adaptation without prompting. These thresholds are product choices to revise after the first test. Record raw counts, not a claim that the entire audience now understands KS. A score above a threshold does not excuse an unrecoverable save error or a misleading explanation.

Test rivalry separately from added difficulty

Run a controlled comparison with the same content and starting conditions: one version retains the chief's initial policy and another applies a recorded lesson. Keep unrelated enemy strength constant. Counterbalance which version participants see first where practical, and reset histories between conditions. Measure whether players notice the difference, change their decisions and describe it as earned or unfair.

Then test persistence over multiple runs. A short demonstration may make learning look exciting while repeated adaptation makes later play exhausting. Track losses, avoidance of rematches, tactical changes and

whether players voluntarily begin another run. Self-reported desire to return is useful formative feedback; it is not retention measured after release.

Question	Evidence	Decision it informs
Do orders change outcomes for understandable reasons	Matched policy results plus player explanations	Tune rules or improve cues, depending on the failure
Does survivor recruitment change behaviour	Recruits and casualties under alternative viable policies	Adjust incentives or clarify the chief objective
Does camp loss feel productive	Correct reset explanations and observed use of a purchase	Rework the first transaction or its timing
Is rivalry perceived as learning	Source-linked rematch, controlled policy comparison and interviews	Keep, constrain or revise each adaptation
Is procedural variety meaningful	Coverage by encounter dimensions plus different successful decisions	Expand authored combinations or repair repetitive output
Is the slice safe to distribute	Packaged build and isolated reopen/corruption checks	Approve a review build or hold the milestone

13 Production plan and ownership

The source review argues for finishing a representative experience before broad content multiplication. Matthew Davis's Into the Breach postmortem shows how readability and combat constraints shaped both weapons and the strategic layer [R4]. KS should apply that discipline within its existing locks: use the slice to determine which production patterns work, rather than redesign the premise whenever an isolated component disappoints.

Work in dependency order

Milestone A is a stable review baseline. Record the latest source and art manifest, isolate test saves, integrate the existing persistence patch after Claude's current pass, and validate both load entry points. Exit evidence is a Windows review build that reopens safely and identifies incompatible history correctly. A local checkpoint of older art does not protect every newer dirty asset; any later backup or commit must name its exact paths and require the existing task authorisation.

Milestone B is a complete reward cycle. Jacob chooses the first Butcher item and the smallest Alpha encounter consistent with the lock. Claude implements the rules and transaction; presentation shows cost, effect, permanence and reward. Exit evidence is an unassisted run in which a chief's reward matters before the player returns to a useful camp. No new tier or tribe production is needed to answer this question.

Milestone C is one evidenced rivalry loop. Select one chief adaptation and one intertribal fight policy. Store the original fight, expose the lesson and verify a changed rematch after reopening. Exit evidence includes a player who can describe the change and the source record that justifies it. More habit types are not the exit criterion.

Milestone D is readability and balance. Run the matched-policy comparisons, inspect the approved 1x presentation and correct failures that prevent understanding. Add the minimum coherent sound cues and complete named missing art. Exit evidence includes the formative study, representative captures and a documented answer to the spam policy question.

Milestone E is the review candidate. Freeze the slice, test the packaged build away from the active editor, record installation and reopening behaviour, and assemble a short truthful demonstration of the loop. Broader roster production begins only when the accepted patterns can be reused without reopening their fundamentals.

The 8 December slice date is 58 calendar days after this paper's date. That is a planning horizon, not evidence that these milestones fit the available labour. Obtain actual throughput from the first complete feature and asset family, reserve time for defects and owner review, and reforecast weekly. If progress slips, narrow optional variants and presentation flourishes before silently dropping the locked recruitment, camp or Alpha payoff.

Coordinate without colliding with Claude

Claude's current active area includes the CampaignArt files, runtime art resources and Windows build work. Jacob's instruction to leave those areas alone during that pass is more specific than a general frontend ownership label. Codex should not edit them concurrently, run another Unity process on the project, or treat this paper as authorisation for integration.

Safe parallel work consists of research, named asset reviews, independent evidence inspection and isolated tasks with agreed file boundaries. The earlier persistence pass already demonstrates the model: separate checkout, narrow backend override, exact patch, tests and a handoff. Once Claude records readiness, one integrator applies the agreed change and owns the combined build. The shared log records what changed, what was checked and what remains uncertain.

Work package	Decision or owner needed	Output	Acceptance evidence
Save integration	Claude after the active pass	Hook up existing save and load APIs	Isolated player recovery and reopen checks
First forged item	Jacob selects effect and persistence; Claude owns rules	One complete purchase and usable item	Spend, equip, fight, lose, reopen; correct permanent state
Alpha encounter	Jacob selects the first threat; Claude owns encounter rules	A post-Skullpeak fight that uses the earned reward	Clear tells, at least one feasible answer and a coherent run ending

Work package	Decision or owner needed	Output	Acceptance evidence
Order balance	Jacob selects intended tradeoff; Claude owns tuning	Comparable policy test and targeted change	Spam advantage understood, fixes measured per encounter
Learning history	Claude owns learning; frontend work scheduled explicitly	One inspectable lesson and changed rematch	Provenance, repeatability and player understanding
Art completion	Jacob approves looks; animation work remains separately scoped	Named missing actions and characters	Native-scale review followed by runtime verification

14 Estimate content cost before expanding it

Budget by accepted deliverables. Record concept attempts, accepted base sprites, required directions, distinct actions, cleanup time, integration time and rejection reasons. Generation credits are only one part of cost; a cheap output that takes repeated repairs or cannot be read in combat can be more expensive than a carefully constrained request.

For planning, multiply the number of approved character variants by required directions and distinct actions, then add props, effects and exceptions. As an illustration only, 18 variants with two directions and eight unique actions imply 288 direction-action clips before alternate equipment. This is not an inventory of KS and is not a request to generate 288 clips. Reuse should be counted only where the result remains appropriate for that character and action.

Use measured ranges from accepted work to estimate the next batch. If a representative set needs more cleanup than expected, update the forecast before authorising the rest. Track the ratio of accepted outputs to attempted outputs and the time from approval to verified use. These numbers diagnose the pipeline better than total frames generated.

Prioritise the assets that communicate a decision: a chief's tell, the Warden's protection, a clear death ending, the player's selected target and the first forged item. Decorative alternatives should not consume the same urgency as a missing opponent in a required encounter. Review-only assets remain outside production until their approval and integration status are explicit.

15 Principal risks and decision register

The main risk is evaluating an incomplete loop as though it were complete. If defeat resets the army but camp materials have no meaningful use, a negative reaction cannot tell the team whether the intended progression works. Conversely, adding a purchase does not excuse unreadable or strategically empty battles. The loop must be complete enough to test and each component must still earn its place.

Another risk is mistaking explanation for persuasion. A result panel should not tell a player they made a tactical mistake merely because a scoring heuristic ranked one statistic highest. Show the relevant facts, acknowledge target access and expose optional evidence. When the game makes a counterfactual claim, test it.

Rival learning can become hidden difficulty inflation or self-reinforcing pressure against struggling players. Restrict what changes, expose a clue, preserve an answer and measure repeated runs. Content generation can amplify this problem if it combines counters without considering what the player has unlocked.

The aesthetic risk is inconsistency among attractive individual assets. The approved scale, region identities and Pal silhouettes provide constraints. Validate all of them in the same fight or camp composition. A polished thumbnail can fail at gameplay size; a large stage can still feel cramped if hit areas, labels and effects compete for the same space.

Decision still needed	Recommended question for Jacob	Why it blocks production
First Butcher forge	Which one visible combat change should the first purchase deliver, and what part persists	Determines effect, cost, equipment art and save transaction
First Alpha hunters	What should their first encounter ask the player to do with Skullpeak's reward	Determines encounter rules, enemies, tells and missing art
Unleash tradeoff	Should careless use primarily cost defence timing, nerve control or another selected opportunity	Prevents contradictory tuning and unexplained punishment
War timing	At what campaign transition should an actual tribe war occur	Determines scheduling, history volume and when the player sees consequences
Recruitment floor	Is a minimum of three recruits intended even after heavy casualties	Determines whether careful nerve victory has sufficient value

Parry timing, the complete class-tier schedule and long-term archive retention also need decisions, but they do not all need to precede the first complete slice test. Separate decisions required now from those that can be tested after the core loop has evidence.

16 Conclusions and limits

The videos give KS useful design questions rather than a template to copy. The strongest application is to make its existing systems depend on one another in a way the player can recognise: composition creates options, orders exploit a situation, nerve victory changes recruitment, recruits and chief orders help the next fight, and the camp and rivalry retain a meaningful history.

The next useful proof is a player completing that cycle and correctly describing what changed. The paper recommends a small complete slice, controlled comparisons and named production gates because they can expose whether the premise works before the team multiplies its content cost.

The evidence has limits. The video sample was selected by the user and is small. Most captions are automatic. Creator accounts and developer postmortems have selection and hindsight biases. The supplementary generator source was available as a repository abstract, not its full chapter. The KS checkout is active and dirty, and this review did not rerun the player. No measured player benefit, market demand, universal genre law, patent clearance or visa eligibility follows from these sources. Those questions require their own evidence.

References

User selected videos

[V1] Savallion. 2023, 25 June. Game design deep dive: what makes autobattlers fun? YouTube. English automatic captions reviewed. Source video.

[V2] Skoottie. 2022, 7 June. History of Autobattlers. YouTube. English automatic captions reviewed. Source video.

[V3] Your Friendly Neighborhood Gamers. 2024, 11 December. Autobattlers 101: An Intro To The Genre. Presented by Dylan. YouTube. English automatic captions reviewed. Source video.

[V4] Gamedev Greedable. 2026, 2 October. Every Auto Battler has the Same Problem. YouTube. English automatic captions reviewed. Source video.

[V5] Snoman Gaming. 2019, 9 March. Good Game Design - Tactics Games. YouTube. English captions marked authored or unspecified. Source video.

Supporting developer and research sources

[R1] Hunicke, R., LeBlanc, M., and Zubek, R. 2004. MDA: A Formal Approach to Game Design and Game Research. Framework paper. Author-hosted paper. Used for relationships among mechanics, behaviour and experience, not an empirical claim that a particular design is fun.

[R2] Cadwell, T. 2020, 18 June. Prototype: Building a Game's Substance. Riot Games. Developer article. Used for representative prototypes, audience definition and playtest evidence.

[R3] Caudill, R., and Lim, J. 2023, 21 July. Talking Tactics: Game Analysis Team. Riot Games. Developer article. Used for combined paper design, simulations and playtesting; simulations alone do not establish progression pacing.

[R4] Davis, M. 2019. Into the Breach Design Postmortem. Subset Games, GDC presentation. Presentation slides. Relevant PDF pages include 8, 14, 31-37 and 40-48. Used for constraint-led design and the relationship between combat and the strategy layer.

[R5] Riot_Mort. 2024, 20 February. Dev Teamfight Tactics: Remix Rumble Learnings. Riot Games. Developer reflection. Used narrowly for the distinction between variation between games and unequal resources between players.

[R6] Shaker, N., Smith, G., and Yannakakis, G. N. 2016. Evaluating content generators. In Procedural Content Generation in Games, pp. 215-224. Springer. University repository record and abstract. Full chapter retrieval failed; only the available abstract supports this paper's limited reference to expressive range and player questionnaires.

[R7] Greedable. Fruit Auto Fighters. Developer Steam page, accessed 11 October 2026. Game page. Used for the declared mechanic and unreleased status, not independent effectiveness evidence.

[R8] Konfa Games and tinyBuild. Despot's Game: Dystopian Battle Simulator. Publisher Steam page, accessed 11 October 2026. Game page. Used as a limited check against an overly broad claim about the absence of attrition and sacrifice in the genre.

KS evidence inspected

[L1] Kindred Siege DESIGN_LOCKS.md. Read 11 October 2026, including the latest Presentation owner decisions. Governs counts, genre, run structure, loss, progression and art constraints. Location: C:/Users/jacob/KindredSiege-Clean/DESIGN_LOCKS.md.

[L2] Kindred Siege HANDOFF_LOG.md. Entries dated 30 September to 11 October 2026 consulted selectively. Includes the recovered five-module proposal, current loop, latest world at 1x and open camp, and the isolated persistence handoff. Location: C:/Users/jacob/KindredSiege-Clean/HANDOFF_LOG.md. These are attributed records; latest Claude checks were not independently rerun here.

[L3] Campaign persistence handoff, 11 October 2026. Isolated branch GameDev/campaign-save-safety, repair commit 224924fb5a0538303d97967ff709a7cf54b43c4e. Location: C:/Users/jacob/.codex/scratch/ks-persistence-20261011/HANDOFF.md. Evidence is scoped to its backend tests and stated limits.

[L4] Source inspection, 11 October 2026. Campaign.cs and CampaignRules.cs under Assets/Scripts/Sim/Learning; CampaignArtLoop.cs and CampaignArtWhy.cs under Assets/Scripts/Slice. Used for current save calls, tribe list, recruitment floor, seeded habits and explanation logic. Exact inspected copies and hashes accompany the internal research evidence.

Appendix Practical review checklist

Before approving the next slice build, verify one complete run with the intended art; a real camp spend; the first Alpha encounter after Skullpeak; a useful application of the last chief reward; a rematch linked to a recorded lesson; and a safe reopen. Then obtain a human explanation of the ending condition, the recruitment payoff, what resets and what the rival learned.

Before starting a larger content batch, verify the approved reference and action list, the actual native-scale result, the event timing contract, the integration owner, the available production time and the test that will establish acceptance. Count an asset as completed only after its intended use has been checked.

Before describing the rivalry system publicly, distinguish authored rules from simulated learning, a stored event from an inferred cause, a reproducible replay from a hypothetical alternate outcome, and an isolated test from the packaged player's behaviour. Use the narrowest claim the evidence supports.